Ders 1 Genişletilmiş Notu: Nesneye Dayalı Tasarıma Giriş: Yazılım Kalitesi, Bağlaşım ve Uyum

Ders 1 Genişletilmiş Notu: Nesneye Dayalı Tasarıma Giriş: Yazılım Kalitesi, Bağlaşım ve Uyum

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Kapsamı ve Yaklaşımı
- 2. Kaynaklar ve Ön Koşullar
- 3. Yazılım Kalitesi
- 4. Neden İyi Tasarım? Projelerin Başarısı
- 5. Bağlaşım (Coupling)
- 6. Kalıtımın Ağırlığı
- 7. Uyum (Cohesion) ve Ölçülmesi
- 8. Veritabanı Sorumluluğu: Bağlaşım ile Uyumun Çatışması
- 9. Tasarım Kalıbı Kavramı ve Akademik Kaynaklar
- Örnek Sorular
- Dersin Toparlaması

1. Dersin Kapsamı ve Yaklaşımı

Dersin temel sorusu, nesneye dayalı tasarımın nasıl doğru bir biçimde yapılacağıdır. Amaç bu sorunun yanıtını öğrenmek ve bunu uygulama becerisi kazanmaktır. Ders iki kısımdan oluşur (ekrandan, 00:10:00):

- Kısım 1: Gang of Four tasarım kalıpları. Dönemin büyük bölümü bu kısma ayrılır.
- Kısım 2: Code Smells and Refactoring (birim sınamaları ile). Bu kısım iki-üç hafta sürer.

Hoca iki kısmı şöyle bağladı: tasarım kalıpları işin nasıl doğru yapılacağını gösterir, ama insan yanlışından da öğrenir. Kod kusurları (code smells) nesneye yönelimin yanlış kullanım örneklerini adlandırarak gösterir. Böyle bir yanlışla karşılaşıldığında yapılan tasarım hatası geriye doğru uyumluluk korunarak düzeltilir; refactoring'in tanımı da budur. Kaynak kitaptaki bütün kod kusurları işlenmeyecek, kendi içinde birbiriyle en çok bağlantılı (interconnected) olan kusurlar ve refactoring eylemlerinden oluşan bir seçme küme görülecektir.

Kod kusurlarına örnek olarak yinelenen kod (Duplicate Code) verildi. Çok benzer bir kodun birden fazla yerde bulunması kopyala-yapıştır programlamadan kaynaklanır ve bakım kâbusu yaratır. Bir kodu tam anlamasanız bile aynı kodun tekrarlandığını görüp ona uygun refactoring eylemini, örneğin Extract Method ya da Move Method'u önerebilirsiniz.

1.1 Kalıpların derste işleniş biçimi

Her kalıp önce tanıtılır, sonra bir probleme en az iki farklı biçimde uyarlanır. Birinci uyarlama probleme kafadan gelen, "bugünü kurtaran" çözümdür. Ardından bu çözümde küçük bir değişiklik yapılması gerektiğinde işlerin nasıl sarpa sardığı gösterilir. Sonra tasarım kalıbı uygulanır ve kazanılan avantajlar görülür; bu arada bazı dezavantajlar da ortaya çıkabilir. Gang

of Four kitabı da her kalıbı anlattıktan sonra avantajlar, dezavantajlar ve tartışma bölümü verir.

> **Hocanın vurgusu:** Sihirli değnek çözümü hiçbir zaman yoktur. Her çözümün farklı avantaj ve dezavantajları vardır. Elimizdeki probleme ve yakın zamanda nerede değişiklik beklediğimize göre en uygun çözümü seçmeye çalışırız.

Hoca öğrencilerden, bir kalıbı bilerek ya da bilmeyerek kendi projelerinde kullandılarsa bunu anlatmalarını istedi: kalıp ne kazandırdı, ne kaybettirdi. Bu, dersin daha etkileşimli ve zengin geçmesi için önemlidir.

2. Kaynaklar ve Ön Koşullar

2.1 Kaynak kitaplar

Kısım 1'in temel kaynağı Dörtlü Çete'nin (Gang of Four) 1994'te yayımladığı kitaptır (ekrandan, 00:10:00):

- *Design Patterns: Elements of Reusable Object-Oriented Software*, Erich Gamma vd. (Gang of Four), Addison-Wesley, 1994.

Tasarım kalıpları meslekte çok önemli olduğundan bu konuda yüzlerce ek kitap vardır. Hocanın slaytındaki ek kaynaklar (ekrandan, 00:14:50):

- *Design Patterns Explained: A New Perspective on Object-Oriented Design*, Alan Shalloway ve James R. Trott, Addison-Wesley, 2004 (2. baskı).
- *Head First Design Patterns*, Elisabeth Freeman vd., O'Reilly, 2004.
- *Design Patterns* (Wordware Applications Library), Christopher G. Lasater, Jones & Bartlett, 2006.
- *Design Patterns Java Workbook*, S. John Metsker, Addison-Wesley, 2002.
- *Applied Java Patterns*, S. Stelting ve O. Maassen, Prentice Hall, 2002.

Hoca kalıplar için verdiği örneklerin bir kısmını bu kitaplardan doğrudan ya da uyarlayarak almış, bir kısmını da kendisi oluşturmuştur.

Kısım 2'nin temel kaynağı Martin Fowler'ın kitabıdır:

- *Refactoring: Improving the Design of Existing Code*, Martin Fowler, Addison-Wesley, 1999.

Fowler nesneye yönelim üzerinde çalışan bir pratisyendir; danışmanlık yapar ve kendi web sitesinde bu konularda birçok kaynak sunar.

2.2 Ön koşullar

Birinci gereksinim herhangi bir güncel nesneye yönelik programlama diline hakimiyettir (ekrandan, 00:14:50). Hoca "güncel" sözünü Smalltalk örneğiyle açıkladı: Smalltalk nesneye yönelimin ilk zamanlarında ortaya çıkmış, başka dillerde olmayan ilginç özellikleri olan bir dildir, ama yaygın kullanılmaz. Güncel diller olarak C++, Java ve C# sayılabilir. Derste kod örnekleri Java ile verilecektir. Tasarım ve modelleme temel bir konudur ve bir dilin ek özelliklerine ihtiyaç duymaz. Bazı kalıplarda bazı dil özellikleri çok kullanışlı olabilir, Gang of Four bunu belirtir, ama aynı iş başka dillerle de yapılabilir.

> **Hocanın vurgusu:** Önemli olan temel nesne yönelimini iyi bilmektir. Özellikle kalıtım ve çok biçimliliğin getirdiği avantajlar birçok kalıpta kullanılacaktır; kalıtım ve çok biçimliliği kullanmayan kalıp neredeyse yoktur.

İkinci gereksinim UML sınıf şemalarına hakimiyettir. UML, yazılım sektöründe fiilen (de facto) kullanılan teknik dokümantasyon dilidir ve bu konuda da çok sayıda kitap vardır. Hocanın önerileri (ekrandan, 00:14:50):

- *UML Distilled*, 3. baskı (2003), Martin Fowler, Addison-Wesley. "Distilled" damıtılmış, özütlenmiş anlamındadır. Hoca bu kitabı, yalnız gösterimi değil kavramların modellemede doğru kullanımını da anlattığı, yani nesneye dayalı modellemeyi de bir noktaya kadar öğrettiği için sevdiğini söyledi.
- *UML 2.0 in a Nutshell*, Dan Pilone ve Neil Pitman, O'Reilly.

2.3 UML ve Gang of Four gösterimi

UML'deki U, "Unified" sözcüğünden gelir. UML standardı oturana kadar nesneye yönelik tasarımı şematize etmeye yönelik birden fazla ekol vardı; bunların en güçlü yanları alınıp geliştirilerek UML oluşturuldu. UML standardı tasarım kalıplarından sonra oturmuştur. Gang of Four kitabı UML'den önce yazıldığı için kitaptaki kalıp şemalarının gösterimi güncel standarttan farklıdır. Hoca kalıp şemalarında kitabın şemalarını birebir kullanmış, ilk bir-iki şemada farkları anlatacağını söyledi; kendi örneklerinin hepsinde ise yeni standart kullanılmıştır.

3. Yazılım Kalitesi

Bu konularla uğraşmanın nedeni, sonuçta daha iyi, daha kaliteli bir yazılım ortaya çıkarmaktır. Peki iyi tasarım ya da kaliteli yazılım nedir?

3.1 İyi tasarımın tanımı

Yazılım kalitesinin çok farklı bileşenleri vardır; ISO ve farklı gruplar farklı tanımlar yapmıştır. Tasarım kalıplarına odaklı daha dar bir tanım şudur (ekrandan, 00:40:00):

> Problem alanı hangi düzeyde parçalara ayrılabiliyorsa, o düzeyde parçalara ayrılabilen bir tasarım da "iyi" olarak nitelendirilebilir.

Hoca bu tanımı sevdiğini, ama içinde bir belirsizlik olduğunu söyledi: bu düzey nasıl, neye göre ölçülecek, nasıl tanımlanacak?

3.2 Dış ve iç kalite özellikleri

Hocanın tercih ettiği gruplama basit ve akılda kalıcıdır (ekrandan, 00:40:00):

- Dış kalite özellikleri kullanıcıya yöneliktir: doğruluk, etkinlik (sistem kaynaklarını etkin kullanma), kullanım kolaylığı.
- İç kalite özellikleri programcıya (yazılımı geliştirene) yöneliktir.

İç kalite özellikleri tek tek ele alındı:

- **Yeniden kullanılabilirlik:** Yazılan kodun başka projelerde de kullanılarak geliştirme çabasını ve masrafını azaltması. Hoca, yeniden kullanılabilirliğin getirdiği kazancın çalışmalarda en fazla %30 civarında görüldüğünü aktardı. Bu oran başta ona düşük gelmiş, ama nedeni anlaşılır: yeniden kullanılabilir kod yazmak için olası değişikliklere dayanıklı kod yazılır, yalnız bugünkü gereksinimler değil kısa ve orta vadedeki değişiklikler de düşünülür, sınama daha ayrıntılı yapılır. Bunların hepsi ek çaba ve ek masraftır. Yine de %30 yabana atılacak bir kazanç değildir.
- **İlgi alanlarının ayrılması:** Problemin ilgi alanları ne düzeyde ayrılabiliyorsa, tasarımın da aynı düzeyde sorumlulukları içeren sınıflara ayrılması. Bu, iyi tasarım tanımıyla doğrudan ilişkilidir.
- **Esneklik:** Yapılan tasarımı benzer başka problemlere uyarlamanın kolay olması.
- **Taşınabilirlik:** Belirli bir platform için yazılan yazılımın başka platformda da çalışabilmesi. Hocaya göre bazı dillerin doğrudan derlenmek yerine bir sanal makine üstünde yorumlanacak şekilde tasarlanmasının en önemli nedeni farklı platformlara uyarlanabilme (cross-platform) kolaylığıdır.
- **Okunabilirlik ve anlaşılabilirlik:** Bakım çalışmaları hayatın bir gerçeğidir; bir projede harcanan çabanın rahatça yarısı bakım aşamasında harcanır. Okuyup anlayamadığımız kodun bakımını rahat yapamayız.
- **Sınanabilirlik:** Kodda illaki hatalar çıkacaktır; önemli olan kodun ne kadar kolay sınanabildiğidir. Bir sorunu sınamak için birden fazla noktaya bakmak, farklı yerlerden farklı şeyleri bir araya getirmek gerekiyorsa, büyük olasılıkla sorumluluklar doğru atanmamıştır.
- **Bakım kolaylığı:** Yukarıdakilerin çoğu aynı noktaya varır.

  Hocanın vurgusu: Kod kusurları konusunda bir alanın ya da metodun açık ve net adlandırılması bile bir refactoring eylemi sayılacaktır, çünkü anlaşılabilirliği önemli oranda artırır.

Tasarım kalıpları özellikle bakım aşamasındaki işleri kolaylaştırarak fayda sağlar. Bir kalıbı kurmak ve uygulamak biraz daha fazla çaba gerektirir, ama bu çaba bakım aşamasında fazlasıyla amorti olur.

4. Neden İyi Tasarım? Projelerin Başarısı

Hocanın slaytında "Neden iyi bir tasarım?" sorusuna dört yanıt verildi (ekrandan, 00:51:30):

- Yazılım projeleri önemli oranda başarısızlığa uğramaktadır.
- Etken sayısı şüphesiz fazladır, ancak "tasarımın iyiliği" denetlenebilen bir etkendir.
- Oluşan bir hatayı düzeltmenin bedeli, yazılım hayat döngüsünde ilerlendikçe üstel olarak artmaktadır.
- Yazılım geliştirme çabalarının çoğu bakım aşamasında harcanmaktadır.

4.1 Başarının tanımı

Başarının en katı tanımı, bir yazılım projesinin üç koşulu birlikte sağlamasıdır:

- Belirlenen işlevleri, yani gereksinimler belgesinde belgelenmiş bütün gereksinimleri karşılamak.
- Önceden belirlenen süre içinde tamamlanmak.
- Önceden belirlenen bütçe içinde kalmak.

Bu "üç sacayağı" tanımıyla ulusal ve uluslararası çalışmalarda başarı oranı yaklaşık %50 çıkar; yani iki projeden biri başarısız olur. Proje hayata geçmiş olsa bile bütçeyi aşmışsa katı tanıma göre başarısızdır. Tanım esnetilip üç ayaktan birinde sapmaya izin verilirse (örneğin bütün gereksinimler zamanında karşılanmış ama bütçe aşılmışsa) proje "kısmi başarılı" ya da "zorlanılmış" sayılır; bu tanımla başarı oranları %80'lere çıkar. Hoca, 1960'lardan beri aynı sorunların konuşulduğunu ve üç ayaktan özellikle sürede esneme gerekmesinin çok sık görüldüğünü söyledi.

4.2 Yazılım geliştirme modeline göre başarı

Hoca kendi sunumlarından birindeki grafiği gösterdi; kaynak Dobb's Journal 2013 IT Project Success Survey'dir (ekrandan, 00:46:40). Grafikte her süreç için başarılı, zorlanılmış ve başarısız proje oranları yan yana çubuklarla verilir; "zorlanılmış" bütçe, süre ve işlev yönlerinden en az birinde başarısızlık demektir.

- Geleneksel (ardışıl şelale, waterfall): katı tanıma göre başarı yaklaşık %50. Hocanın okumasına göre başarısız %18, zorlanılmış %32; başarılı ile zorlanılmış birlikte yaklaşık %81 (49 + 32).
- Ad-hoc (belirli bir yöntem olmadan, kişilerin çalışma alışkanlıklarıyla ilerleyen projeler): gelenekselle hemen hemen aynı.
- Çevik (agile): başarılı oranı belirgin biçimde yüksek; hoca bunu önemli bir avantaj saydı.
- Yinelemeli (iterative) süreçler: çevikle benzer.
- Yalın (lean): hocanın bu konuda pek bilgisi olmadığını söylediği, grafikte biraz daha iyi sonuç veren süreç.

4.3 Ülkemizdeki durum

2014'te sunulan ulusal bir çalışmada (E. Çetin, P. O. Durdu, "Türkiye'de Çevik Yazılım Geliştirme Üzerine Bir İnceleme", UYMS 2014) kurumlara, gerçekleştirdikleri çevik projelerin başarı oranı soruldu (ekrandan, 00:50:00). Pasta grafikteki sayılar yanıt veren sayısı, yüzdeler o sayının toplamdaki oranıdır:

Kurumdaki çevik projelerin başarı oranı	Yanıt sayısı	Oran
%10'un altı	2	%6
%10-%25	4	%12
%26-%50	5	%14
%50-%75	13	%37
%76-%100	11	%31

Kurumdaki çevik projelerin başarı oranı	Yanıt sayısı	Oran

Toplam 35 yanıttan 24'ü (13 + 11, yaklaşık %69) kurumunda çevik projelerin yarıdan fazlasının başarılı olduğunu bildirmiştir (ara adım eklendi).

4.4 Çıta neden hep yükseliyor?

Hoca, 1960'lardan beri çaba harcanmasına rağmen tablonun çok değişmemesini şöyle açıkladı: kullanıcılar bilgi sistemlerinin işlerini kolaylaştırdığını gördükçe daha büyük problemlerin çözümü için yazılım projelerine girişirler. Hedef çıta yükseldikçe daha fazla çalışmak gerekir ve geride kalma durumu oluşabilir. Yani ilerleme kaydediyoruz, ama problemlerimiz de sürekli ilerliyor. Bu yüzden daha işin başında adımları sağlam atmak gerekir: tasarım kalıpları gibi bizi iyiye götüren çözümleri bilmek ve ileride bakım sorunu çıkaracak kod kusurlarına erkenden düşmemek.

5. Bağlaşım (Coupling)

İyi tasarıma götüren en temel iki ilke, en önemli iki iç kalite ölçüsü olan bağlaşım ve uyumdur. Bağlaşımın düşük, uyumun yüksek olması istenir: low coupling, high cohesion. İlgi alanlarının ayrılması ilkesi de vardır, ama bu ikisine uyulduğunda o da özellikle uyuma yakın olarak kendiliğinden sağlanır.

5.1 Tanım

Slayttaki tanım: bağlaşım, bir parçanın diğer parçalara bağımlılık oranıdır (ekrandan, 01:07:30). Hoca, tanımın içinde tanımlanan kavrama yakın bir sözcük ("bağımlılık") kullanmanın yumurta-tavuk döngüsü gibi olduğunu, bu yüzden "bir parçanın sistemdeki diğer parçalarla ilişkide olma oranı" demenin daha iyi olduğunu söyledi. Parça sınıf, alt sistem ya da paket olabilir; nesneye dayalı tasarımda en somut düzey sınıftır ve derste örnekler bu düzeyde verilir.

Bir sınıfın diğerine bağımlı olması şu durumlarda söz konusudur (ekrandan, 01:07:30):

- Diğerinin hizmetlerinden yararlanması,
- İç yapısından haberdar olması,
- Çalışma prensiplerinden haberdar olması,
- Onun özelleşmiş ya da genelleşmiş hali olması (kalıtım ilişkisi).

Oklar yönlüdür; tek yönlü ya da çift yönlü ilişki olabilir. İki sınıf arasında herhangi bir ok varsa bağlaşım, yani bağımlılık vardır. Önemli olan ilişkinin niteliği değil, varlığıdır; ilişkide olunan sınıfların sayısı arttıkça bağlaşım artar. Düşük bağlaşım yeniden kullanılabilirliği artırır ve bir sınıfta yapılan değişikliğin diğer sınıflarda yan etki çıkarma olasılığını düşürür.

5.2 UML gösterimi ve Violet UML

Hoca şemaları Violet UML adlı küçük bir araçla çizdi (ekrandan, 01:10:00). Araç bir jar dosyasıdır, özel kurulum gerektirmez, Java çalışma ortamı (JRE) kurulu bilgisayarda çalışır; kullanım durumu (use case), sınıf, nesne, durum (state), etkinlik (activity) ve sıralama (sequence) şemaları çizilebilir ve şemalar resim olarak dışa aktarılabilir.

Derste hatırlatılan gösterimler:

- Düz çizgi ve ucu açık ok sahiplik (association) ilişkisidir. A'dan B'ye böyle bir ok, A sınıfında B türünden bir üye alan bulunduğu anlamına gelir; örneğin `- b : B`. Eksi işareti private demektir.
- Nicelik (multiplicity) okun uçlarına yazılır. * sıfır ya da daha fazla demektir; A'da B türünden bir dizi ya da jenerik liste vardır (hocanın deyişiyle çoklu sahiplik). Alt sınır gerekiyorsa iki nokta kullanılır: `1..*` en az bir demektir, sola alt sınır, sağa üst sınır yazılır. Nicelik yazılmamışsa iki taraf da 1 kabul edilir.

- Kesikli çizgi sahip olmadan kullanma ilişkisidir; örneğin bir metot içinde geçici bir nesne oluşturup kullanmak.
- Bir şema bileşeni varsayılan anlamı dışında kullanılıyorsa anlamı «…» stereotip işaretiyle belirtilir.

  Hocanın vurgusu: A ile B arasında birebir ya da bire çok ilişki olması bağlaşımı değiştirmez. A'dan B'ye ok var mı? Varsa bağlaşım vardır.

5.3 İki tasarımın karşılaştırılması

Aynı amaca hizmet eden iki tasarım çizildi (ekrandan, 01:20:00). Bütün nicelikler 1 kabul edildi:

```
Tasarım I:  A ---> B ---> C        (A, B'ye; B, C'ye bağlı)
Tasarım II: A ---> B
            A ---> C               (A hem B'ye hem C'ye bağlı)
```

Hoca sınıfa hangisinin tercih edilmesi gerektiğini sordu. Bir öğrenci tasarım II'yi seçti: tasarım II'de B ile C'nin hiçbir bağımlılığı yoktur.

Önce her sınıfın bağlaşımı sayıldı. Okun yönü önemlidir; B, A'ya bağımlı değildir. Çift yönlü bağımlılık olsaydı 2 sayılacaktı.

```
Tasarım I              Tasarım II
bağ(A) = 1             bağ(A) = 2
bağ(B) = 1             bağ(B) = 0
bağ(C) = 0             bağ(C) = 0
topBağ(I) = 2          topBağ(II) = 2
maxBağ(I) = 1          maxBağ(II) = 2
```

Sistemin bütünü olarak bakıldığında iki tasarımın toplam bağlaşımı da 2'dir; birini öbüründen bariz üstün kılan bir şey yoktur. Öğrencinin işaret ettiği avantaj gerçektir: tasarım II'de B'yi başka bir projede kullanmak isteyen yalnız B'yi alır. Tasarım I'de B'yi yeniden kullanmak isteyen C'yi de almak zorundadır, çünkü B, C olmadan kullanılamaz.

Hoca ardından maksimum bağlaşım ölçütünü önerdi ve bunu riskli kararlarda kullanılan bir stratejiye benzetti: olası bütün senaryoların sonuçlarına bakılır ve hepsinde kaybediliyorsa en az kayıpla kapatılacak senaryo seçilir. Bu ölçüte göre tasarım I'in en kötü sınıfı 1, tasarım II'nin en kötü sınıfı (A) 2 bağlaşıma sahiptir; bu kez tasarım I daha iyi görünür.

> **Ders sonrası not:** Hoca bu karar stratejisini (galiba diyerek) John Nash'in sıfır toplamlı oyunlar üzerindeki çalışmalarıyla ilişkilendirdi. Sıfır toplamlı oyunlarda en kötü durumu en iyileyen minimax stratejisinin temel teoremi büyük olasılıkla John von Neumann'a aittir; Nash'in en bilinen katkısı Nash dengesidir.

5.4 Dolaylı bağlaşım

Analiz daha da ayrıntılandırıldı. Tasarım I'de A, B'ye; B de C'ye bağlıysa A dolaylı olarak C'ye de bağlıdır: A'yı kullanmak için B'yi, B'yi kullanmak için de C'yi almak gerekir. Hoca bunu şemada A'dan C'ye kesikli bir çizgi ve «dolaylıBağ» stereotipi ile gösterdi (ekrandan, 01:23:40). Dolaylı ilişkiyi doğrudan ilişki gibi 1 saymak haksızlık olur; hoca yarım (0,5) saymayı önerdi:

```
Tasarım I (dolaylı dahil)   Tasarım II (değişmez)
bağ(A) = 1 + 0,5 = 1,5      bağ(A) = 2
bağ(B) = 1                  bağ(B) = 0
bağ(C) = 0                  bağ(C) = 0
topBağ(I) = 2,5             topBağ(II) = 2
maxBağ(I) = 1,5             maxBağ(II) = 2
```

Tasarım II'de dolaylı ilişki olmadığı için sonuçlar değişmez. Artık toplam bağlaşıma göre tasarım II, maksimum bağlaşıma göre yine tasarım I avantajlıdır.

Sonuçta bu sorunun net bir yanıtı yoktur; iki tarafın da önemli savunma noktaları vardır. Hocanın asıl vurgusu şuydu: tasarım bir problemi çözmek için yapılır. A, B, C varlıklarının problem

alanındaki ilişkisi nasılsa tasarım da onu yansıtmalıdır. Varlıklar arasındaki ilişki tasarım I'deki gibiyse, hangi avantaj için olursa olsun tasarım II'yi yapmak yanlıştır.

> **Hocanın vurgusu:** Bir şeyi ölçerken kullanılan ölçütler literatürde çok fazladır ve hepsinin ince ayrıntıları vardır. Kullanılan ölçüte göre ölçüm sonuçları değişir.

6. Kalıtımın Ağırlığı

Bağlaşım hesabında kalıtım ayrıca ele alındı. Kalıtım, sahiplik ya da parça-bütün ilişkilerinden daha "ağır sıklet" bir ilişkidir, çünkü bir kez kurulduğunda değiştirilemez. Sahiplik ilişkisinde belirli bir A nesnesine önce B1 nesnesi bağlanmışsa sonra yerine B2 konabilir.

Hocanın örneği Kişi ve Çalışan sınıflarıdır. Çalışan, Kişi'den kalıtım alıyorsa her çalışan her zaman bir kişidir. Bir kişi için Çalışan nesnesi oluşturulmuşsa, o kişi emekli olduğunda nesneyi Çalışan olmaktan Kişi olmaya rahatça taşıyamazsınız. Kişiye ait bilgileri alıp yeni bir Kişi nesnesi oluşturmanız gerekir. Bu ek iş demektir ve yeni nesnenin kimliği (evrensel tekil tanımlayıcısı, UUID) farklı olur; bellekte bambaşka bir nesnedir.

Bu nedenle bağlaşım hesabında kalıtım ilişkisi 1,5 ya da 2 gibi bir katsayıyla sayılabilir; bu da bütün hesapları değiştirir.

> **Hocanın vurgusu:** Tasarım kalıplarında hatalı çözüm örneklerine bakarken, işlerin genelde sınıflar arasındaki kalıtım ilişkisi yanlış kurulduğunda sarpa sardığını göreceğiz.

7. Uyum (Cohesion) ve Ölçülmesi

7.1 Tanım ve yararları

Uyum, bir parçanın sorumluluklarının birbirleriyle uyumlu olma oranıdır (ekrandan, 01:30:00). Hoca burada da tanımda aynı sözcüğün geçtiğini belirterek şöyle de ifade etti: bir parçanın sorumluluklarının birbirine yakın alanlarda olması, birbirine zıt görevler içermemesi.

Yüksek uyumun yararları (ekrandan, 01:30:00):

- **Sınıfın anlaşılma kolaylığı artar.** Sorumlulukları birbirine yakın bir sınıf kolay anlaşılır; sorumluluklar farklıysa sınıfı anlamak için farklı konularda bilgi birikimi gerekir.
- **Yeniden kullanılabilirlik artar.** Yüksek uyumlu sınıf, belirli bir problemin çözümüne odaklanmış ve uzmanlaşmış bir sınıftır; aynı ya da benzer problemle karşılaşıldığında yeni projede rahatça kullanılır.
- **Bakım kolaylığı artar.**
- **Sınıfın değişikliklerden etkilenme olasılığı düşer.** Uyumu yüksek sınıf değişen gereksinimler karşısında küçük bir hedeftir; daha az çeşitlilikte gereksinim karşıladığı için, bir gereksinim değiştiğinde bunun o sınıfın sorumluluklarıyla ilgili olma olasılığı düşüktür.

Düşük bağlaşım getiren bir tasarım genelde yüksek uyum da getirir; yüksek uyumlu bir tasarım da düşük bağlaşım getirir. Kötü yönde de durum aynıdır: yüksek bağlaşım düşük uyumu, düşük uyum yüksek bağlaşımı beraberinde getirir.

7.2 Uyum ölçütleri

Uyumu ölçmek bağlaşımı ölçmekten zordur; uyum ölçütleri önermek ve değerlendirmek daha güçtür. İki yaklaşım vardır:

- Sınıf tasarımı ya da kaynak kod üzerinden, yani statik kod analiziyle hesaplanan ölçütler.
- Kaynak koddaki yorumları dilbilimsel (lingüistik) olarak inceleyen, doğal dil işleme ve makine öğrenmesi yöntemleri kullanan ölçütler.

Hoca, kendi bildiği kadarıyla ikinci grubun başarısının kod analizine dayalı ölçütlerden çok farklı olmadığını, ama literatürü güncellemesi gerektiğini söyledi.

7.3 Metot-alan grafı örneği

Kod analizine dayalı birçok uyum ölçütü şu fikre dayanır: metotların sınıfın üye alanlarını kullanma yoğunluğu uyumla doğru orantılıdır. Hoca bir sınıfın alanlarını ve metotlarını PowerPoint'te çizdi (ekrandan, 01:40:00):

- Üye alanlar: a1, a2, a3, a4.
- Üye metotlar: m1, m2, m3, m4, m5.
- Kullanım: m1 → a1, a2; m2 → a2; m3 → a2, a3; m4 → a4; m5 → a4.

Verilen: 4 alan, 5 metot, 7 kullanım kenarı.

İşlem: Her metot her alanı kullansaydı, yani graf tam bağlı (fully connected) olsaydı en fazla 4 × 5 = 20 ilişki olurdu. Uyum, var olan ilişkilerin bu en büyük sayıya oranıdır:

$$\text{Uyum} = \frac{7}{20} = 0{,}35$$

Böyle bir oran her zaman 0 ile 1 arasında çıkar: tam bağlıysa 1, hiç ilişki yoksa 0. Bu yüzden yüzde olarak kolayca yorumlanır.

Yorum: %35 düşük bir değerdir. Graf hiçbir kenar kesilmeden ikiye bölünebilir: {a1, a2, a3, m1, m2, m3} ve {a4, m4, m5}. Bu, sınıfa büyük olasılıkla birbirinden uzak iki sorumluluk yüklendiğini gösterir. Sınıf iki sınıfa bölündüğünde:

- Birinci sınıf: 3 alan, 3 metot, en fazla 3 × 3 = 9 ilişki; var olan 5 ilişki. Uyum $5/9 \approx 0{,}56$.
- İkinci sınıf: 1 alan, 2 metot, en fazla 2 ilişki; ikisi de var. Uyum $2/2 = 1{,}00$.
- Ortalama: $(0{,}56 + 1{,}00)/2 = 1{,}56/2 \approx 0{,}78$.

Sınıfı ikiye bölmek uyumu 0,35'ten yaklaşık 0,78'e çıkardı. Bu bölme işlemi de aslında bir refactoring eylemidir. Bir sınıfa iki ayrı sorumluluk yüklemek yerine iki ayrı sınıfa birer odaklanmış sorumluluk verildiğinde sonuç çok daha iyidir.

> **Ders sonrası not:** Hoca ekrandaki kutulara "Bağ=7/20 = 0,35", "Bağ=5/9 = 0,56", "OrtBağ=0,78" yazdı ve anlatırken de "bağlaşım" dedi; ardından kendisi bunun dil sürçmesi olduğunu söyledi. Bu değerlerin hepsi uyum (cohesion) değeridir.

8. Veritabanı Sorumluluğu: Bağlaşım ile Uyumun Çatışması

8.1 Problem

Önceki tartışmada varlıklar arasındaki ilişkinin tasarım I'deki gibi (A→B→C) olduğunu ve bu tasarımın seçildiğini varsayalım. Bu varlıkların bilgileri bir veritabanında ya da serileştirilip bir disk dosyasında tutulacaktır. Okuma ve yazma sorumluluğu kime verilmelidir? İki seçenek çizildi (ekrandan, 01:46:45 ve 01:50:00):

```
Seçenek 1: Her nesne kendini yazar/okur
  A ---> B ---> C
  A, B, C ....«DB ops»....> «Database»

Seçenek 2: Ayrı bir DbOps sınıfı (D)
  A ---> B ---> C
  D ....> A,  D ....> B,  D ....> C     (sahip olmadan kullanma)
  D ....«DB ops»....> «Database»
```

D sınıfı A, B, C nesnelerini veritabanına yazma ve oradan okuma sorumluluğunu taşır; bu nesnelere sahip olmasa bile onları kullanan metotlar içerir. Kesikli çizgi sahip olmadan kullanma ilişkisini gösterir.

8.2 Sınıfın yanıtları ve bağlaşım hesabı

Bir öğrenci ikinci seçeneğin daha hızlı çalışacağını, çünkü veritabanı bağlantısı ne kadar az açılırsa o kadar verimli olacağını söyledi; hoca her yeni bağlantının bir maliyeti olduğunu belirterek bunu doğru buldu.

Sonra bağlaşım hesaplandı. Dolaylı ilişkiler sayılmazsa seçenek 1'in toplam bağlaşımı 2, maksimum bağlaşımı 1'dir. Seçenek 2'de D'den A, B ve C'ye ilişkiler eklenir. Bunlar tam sahiplik değil, yalnız kullanma ilişkisi olduğu için yarım sayılabilir:

```
Seçenek 2
bağ(A) = 1
bağ(B) = 1
bağ(C) = 0
bağ(D) = 3 × 0,5 = 1,5
topBağ = 3,5
maxBağ = 1,5
```

Toplam bağlaşım 2'den 3,5'e çıktı; yani yaklaşık %75 arttı ($3{,}5/2 = 1{,}75$). Bu sırada sohbet penceresinden bir katılımcı, soldaki çözümün bağlaşımı daha az ama uyumunun da az olduğunu yazdı. Hoca bunun tam doğru noktaya parmak basmak olduğunu söyledi: seçenek 1'de uyum ilkesi göz ardı edilmiştir.

8.3 Değişiklik senaryosu

Veritabanı olarak Microsoft SQL Server kullanıldığını, lisans maliyetleri arttığı için MySQL ya da PostgreSQL'e geçilmek istendiğini düşünelim. İkisi de standart SQL ile erişilse de gerçek hayatta bazı kod değişiklikleri illaki gerekir (saklı yordamlar kullanıldıysa veritabanı tarafında daha da çok değişiklik olur; burada nesne tarafına odaklanılır). Seçenek 1'de veritabanı koduyla ilgili değişiklik bütün sınıflarda tekrarlanmalıdır. Seçenek 2'de yalnız D sınıfını değiştirmek yeterlidir.

> **Hocanın vurgusu:** Akıllı insan bir işi bir defa yapar. Bir iş için birden fazla yerde değişiklik yapmak, yanlış bir tasarım kararı verildiğinin göstergesidir.

Yeni bir sorumluluk olan veritabanı işlemlerini ayrı bir sınıfta gerçeklemek doğru karardır; bağlaşım artmış olsa da doğru çözüm seçenek 2'dir. Bu çözüm aynı zamanda katmanlıdır: A, B, C sınıfları iş mantığı katmanını, D sınıfı veritabanı katmanını oluşturur (ekrandan, 01:50:00). Hoca bunu, uyumu daha yüksek olan çözümü seçmenin her zaman yararımıza olduğu biçiminde bağladı.

8.4 Soru-cevap

Soru: İkinci modelin doğru olduğunu anladık. Ama düşük bağlaşım ve yüksek uyum ölçütlerine baktığımızda, bağlaşım yüksek olsa bile uyum yüksekse mutlaka o tarafa mı yönelmeliyiz? Uyum her durumda bağlaşımı geçiyor mu?

Cevap: Evet, uyum biraz daha ağırlıklıdır. İyi tasarım tanımında problem hangi düzeyde parçalara ayrılıyorsa tasarımın da o düzeyde parçalanması gerektiği söylenmişti. Bu düzeyin ne olduğu bazı durumlarda kolay anlaşılmaz. Sistem düzeyinde bakıldığında her probleme özgü "olması gereken" bir bağlaşım düzeyi vardır. Seçenek 1'in bağlaşımı daha düşüktür, ama bu problemde olması gereken bağlaşım seçenek 2'deki düzeydedir; bağlaşımı bunun altına indirmek iyi bir iş değildir. Bağlaşım ile uyum arasında çelişki olduğunda uyum tarafından gitmek tercih edilir.

9. Tasarım Kalıbı Kavramı ve Akademik Kaynaklar

9.1 Terimin kökeni

"Design pattern" terimi mimarlıktan gelmiştir. Bilgisayar mühendisliği ve özellikle yazılım mühendisliği nispeten yeni bir disiplindir ve daha eski disiplinlerden terim ödünç alır. Hocanın verdiği mimari kalıp örneği: içinde en az bir kişinin bir saatten fazla zaman geçireceği her oda mutlaka güneş ışığı alan bir pencereye sahip olmalıdır. Mimarlıkta tasarımcıyı yönlendiren bu tür şablonlara, örüntülere kalıp denir. Yazılımda da tasarım kalıpları sık karşılaşılan modelleme problemlerinin iyi çözümüne yönelik önerilerdir.

> **Ek bilgi (derste anlatılmadı):** Mimarlıktaki kalıp dili fikri büyük olasılıkla Christopher Alexander'ın 1977 tarihli *A Pattern Language* kitabına dayanır. Gang of Four kitabının giriş bölümü de kalıp tanımını Alexander'dan alıntılayarak başlar.

9.2 Kalıpların kategorileri ve kararlılık

Hoca IEEE Xplore'da tasarım kalıplarıyla ilgili yayınları tararken iki konuya dikkat çekti. Birincisi, kalıpları uygulamadan önce kaynak kodu inceleyerek kalıp adaylarını tespit etme konusudur. İkincisi "Do Structural Design Patterns Promote Design Stability?" başlıklı makaledir (COMPSAC'06, IEEE, 2006) (ekrandan, 02:20:00). Tasarım kalıpları kendi aralarında üç kategoriye ayrılır: yaratımsal (creational), yapısal (structural) ve davranışsal (behavioral). Makale, yapısal kalıpları uygulamanın tasarımı daha kararlı (stabil) yapıp yapmadığını sorar. Bir tasarım, zaman içinde, bakım aşamasında ne kadar az değişikliğe uğruyorsa o kadar kararlıdır. Makalede incelenen Adapter, Bridge, Composite ve Facade, ilerleyen haftalarda tek tek görülecek kalıplardır.

Ekrandaki makale sayfasında kararlılık için literatürden şu bakış açısı benimsenir: yazılım kararlılığı, yazılım değiştirildiğinde değişikliklerin yayılmasına (potansiyel dalga etkisi, ripple effect) karşı dirençtir. Bir sınıftaki değişiklik diğer sınıflarda dalga etkisi yaratabilir; bu etki istenen (ek değişiklik gerektirmeyen) ya da istenmeyen türde olabilir (ekrandan, 02:20:00).

9.3 Akademik veri tabanları

Hoca, dergi makaleleri, konferans bildirileri ve e-kitaplar için üniversitenin abone olduğu elektronik veri tabanlarını tanıttı. Bilgisayar mühendisliği açısından önemli olanlar:

- ACM (Association for Computing Machinery) Digital Library.
- IEEE Xplore: hem dergi hem konferans yayınları; IEEE mesleğin önemli bir kuruluşudur.
- ScienceDirect: mühendislik odaklıdır ve önemli bir bilgisayar mühendisliği koleksiyonu vardır.
- Springer: özellikle e-kitap ve konferans kitapları.
- Wiley: hem dergi hem kitap.
- Web of Science: özet araması yapar; diğerleri tam metin sunar.

IEEE Xplore'da arama yaparken yalnız abone olunan (subscribed) içeriği gösterme seçeneği, tam metne erişilebilecek yayınları süzer. Yayın türleri arasında "Early Access", yayına kabul edilmiş ama henüz basılmamış, yalnız çevrim içi yayımlanmış makale demektir. Konu filtreleriyle (object-oriented, software architecture, software maintenance, Java, UML gibi) "design" sözcüğünün elektronik ya da anten tasarımı gibi alakasız sonuçları ayıklanabilir.

Hoca ayrıca bir telif uyarısı yaptı: indirilen PDF'lerde indiren kurumu ve indirme zamanını gösteren görünür ve görünmez filigranlar vardır. Bu yayınlar yalnız aynı kaynaklara erişim hakkı olan kişilerle (örneğin abone başka bir üniversitedeki akademisyenle) paylaşılabilir; erişim hakkı olmayan birine göndermek telif haklarının ihlalidir.

Örnek Sorular

Soru 1: Hangi tasarım? (ekrandan, 01:20:00 ve 01:23:40)

Aynı amaca hizmet eden iki tasarım veriliyor. Tasarım I: A→B, B→C. Tasarım II: A→B, A→C. Bütün nicelikler 1'dir. Bağlaşım açısından hangisi tercih edilmelidir?

Derste verilen çözüm:

- Doğrudan bağlaşım: Tasarım I'de bağ(A) = 1, bağ(B) = 1, bağ(C) = 0, toplam 2, maksimum 1. Tasarım II'de bağ(A) = 2, bağ(B) = bağ(C) = 0, toplam 2, maksimum 2.
- Toplam eşit; maksimuma göre I iyi; yeniden kullanılabilirlik açısından II'de B ve C bağımsızdır.
- Dolaylı bağlaşım 0,5 sayılırsa Tasarım I: bağ(A) = 1,5, toplam 2,5, maksimum 1,5. Toplamda II, maksimumda I üstündür.
- Net yanıt yoktur; belirleyici olan, varlıklar arasındaki ilişkinin problem alanında nasıl olduğudur.

Soru 2: Veritabanı sorumluluğu kimde olmalı? (derste sözlü, 01:44:23; ekrandan, 01:50:00)

A→B→C tasarımında varlıklar bir veritabanında saklanacaktır. Okuma ve yazma sorumluluğu nesnelerin kendisine mi verilmeli, yoksa bu iş için yeni bir sınıf (DbOps) mı oluşturulmalı?

Derste verilen çözüm:

- Yeni sınıf D, A, B, C'yi kullanır (her biri 0,5): bağ(D) = 1,5, toplam bağlaşım 2 → 3,5 (yaklaşık %75 artış), maksimum 1,5.
- Buna karşın uyum yükselir: veritabanı değişirse yalnız D değişir; seçenek 1'de değişiklik bütün sınıflarda tekrarlanır.
- Doğru çözüm ayrı sınıftır; ayrıca iş mantığı ve veritabanı katmanlarını ayıran katmanlı bir çözümdür.

Soru 3: Uyum ile bağlaşım çelişirse? (derste sözlü, 01:50:57)

Bağlaşım yüksek olsa bile uyum yüksekse her zaman o tasarıma mı yönelmeliyiz?

Derste verilen yanıt: Uyum daha ağırlıklıdır. Her problemin olması gereken bir bağlaşım düzeyi vardır; bunun altına inmek iyi bir iş değildir. Çelişki durumunda uyum tarafı tercih edilir.

Dersin Toparlaması

Bu ilk derste dersin iki kısmı tanıtıldı: Gang of Four tasarım kalıpları ve Fowler'ın kitabına dayanan kod kusurları ile refactoring. İyi tasarım, problemin parçalandığı düzeyde parçalanan tasarım olarak tanımlandı. Kalite ölçütleri kullanıcıya yönelik dış ve geliştiriciye yönelik iç özellikler olarak ayrıldı. Yazılım projelerinin katı tanıma göre yaklaşık yarısının başarısız olması, bakımın çabanın büyük kısmını oluşturması ve hata düzeltme bedelinin hayat döngüsünde üstel artması, iyi tasarımın neden önemli olduğunu gösterdi.

Dersin merkezinde iki temel ilke vardı: düşük bağlaşım ve yüksek uyum. Bağlaşım, sınıflar arası okların sayısı üzerinden toplam, maksimum ve dolaylı ölçütlerle hesaplandı; ölçüt değişince tercih edilen tasarımın da değişebildiği görüldü. Uyum, metotların alanları kullanma yoğunluğu ile ölçüldü; iki sorumluluğu taşıyan bir sınıfı bölmenin uyumu 0,35'ten yaklaşık 0,78'e çıkardığı gösterildi. Veritabanı örneğinde bağlaşım artsa da uyumu yüksek ve katmanlı çözümün doğru olduğu sonucuna varıldı. Gelecek haftadan itibaren tasarım kalıplarının kendisine geçilecektir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.